Production Incident Triage and Runbook Assistant

Project Proposal and Design Plan | ML System Design and AgentOps (UE24AM342AA5) | Mini Project 08

PES University, Department of CSE (AI & ML)

Status: design stage. No component is implemented and no measurements exist yet.

1. Project Overview

An on-call engineer paged at night receives a *symptom* (disk filling, replication lag, latency spike), not a cause. Under time pressure and fatigue they must identify the problem class, find the applicable runbook, set severity and decide whether to escalate. Documentation is organised by cause while alerts arrive as symptoms, so a single embedding search often misses the relevant provision. The planned system is a multi-agent service for on-call engineers that validates an alert and on-call note, retrieves PostgreSQL guidance for one chosen major version, proposes a cause class and first-response steps with citations, and routes the incident. It never executes remediation; the engineer decides.

2. Planned System Architecture

```
Web client -> FastAPI (validate; quarantine rejects) -> Queue -> run_id returned to caller
   -> Agent workers (scalable): [Triage Agent] -> [Retrieval & Verification Agent]
        tools: corpus search | clause lookup | request-field lookup
        Vector index (FAISS/Chroma) <- CLI indexing pipeline
   -> Result store -> status endpoint (polling)
Observability: Prometheus, Grafana, OpenTelemetry -> Jaeger/Tempo, structured logs
```

- **Triage Agent:** classifies the symptom (saturation, contention, configuration, dependency failure, data-volume), extracts entities and proposes severity and route.
- **Retrieval & Verification Agent:** calls retrieval tools iteratively, checks the evidence, and retries with a different strategy or escalates.
- **Deployment boundaries (Docker Compose, at least four services):** API, worker, queue, vector store and monitoring run separately, so workers scale independently and failures stay isolated. Execution is asynchronous because runs take tens of seconds; the caller receives a run ID and polls.

3. Planned Implementation

- **Stack:** FastAPI, LangGraph or CrewAI, LangChain or LlamaIndex, FAISS or Chroma, local sentence-transformers embeddings, Redis, Prometheus, Grafana, OpenTelemetry, Locust, TruLens or equivalent (final choices to be confirmed).
- **Ingestion:** a published data contract with schema, range, categorical and cross-field validation. Invalid requests are quarantined and never start an agent run.
- **Corpus:** PostgreSQL documentation (monitoring, WAL, replication), three self-written runbooks and a severity/escalation policy.
- **Iterative retrieval:** retrieve the apparent provision, search for provisions that modify it, re-query from what was read, then verify support. The reference case (disk growing while table and index sizes are stable) should reach the replication-slot WAL retention provision.
- **Verification and escalation:** every citation must resolve to retrieved text; unverified answers are not returned. The system escalates to a human when several cause classes have similar support or the symptom is outside the corpus.
- **Routing:** self-serve runbook, page-secondary or incident-commander, with the basis stated.
- **Safeguards:** prompt-injection defence, output filtering, toxicity checks, timeouts, exponential-backoff retries, a circuit breaker with a fallback path and bounded agent loops.
- **Observability:** correlation IDs and tracing so any run can be reconstructed (tool calls, passages, prompt version, latency, cost).

4. Planned Evaluation

- **Evaluation set:** at least 20 scenarios, each with an expected outcome and supporting passage, including the replication-slot case, run as a CLI batch.
- **Metrics:** groundedness, context relevance, faithfulness and hallucination rate, compared across two prompt variants and before and after one deliberate change.
- **Performance:** a p95 SLO declared before measurement, then Locust load tests with a stub LLM measuring RPS, p50, p95, failure concurrency and the effect of `--scale=3`. Cost per run tracked per step.
- **Results:** none yet; they will be reported only after measurement.

5. Conclusion

The project targets the inverted-retrieval problem in incident response by combining agentic multi-step retrieval, mandatory evidence verification and explicit escalation. If built as designed, it will give on-call engineers cited, auditable recommendations while keeping decisions with humans, preferring a safe "I don't know" to a fabricated answer.